SQL AND RELATIONAL DATABASES

Creating and Using Relational Databases with SQL

A step-by-step guide to querying business data

Tables Joins Aggregation Set operations

Practical Web Database Design
Chapter 3: Creating and Using Relational Databases with SQL

Presented by Abdalla Fadl
Full-Stack Java Developer
youtube.com/c/AbdallaFadl

02

Chapter roadmap

Table definitions and constraints
Inserting, updating, and deleting data
Filtering, sorting, and joining rows
Aggregation, aliases, and set operations
Building and testing a complete query

Building and testing a complete query

SQL is declarative

A query describes the required result.
The DBMS chooses a physical execution plan.
SQL dialects differ across database products.
Applications send SQL through database drivers.

Applications send SQL through database drivers.

SQL operations

Purpose	Commands
Define structures	CREATE TABLE, ALTER TABLE, DROP TABLE
Change data	INSERT, UPDATE, DELETE
Retrieve data	SELECT

The example database

CUSTOMER identifies customers and their credit limits.
SALE records sale dates, amounts, and customer references.
PRODUCT records prices and stock levels.
EMPLOYEE can reference another employee as manager.

EMPLOYEE can reference another employee as manager.

CREATE TABLE

Columns define the structure. Constraints define valid data.

```
CREATE TABLE CUSTOMER (
  CustomerNo VARCHAR(9) PRIMARY KEY,
  First VARCHAR(40),
  Last VARCHAR(40) NOT NULL,
  CreditLimit DECIMAL(12,2) DEFAULT 0 NOT NULL,
  CHECK (CreditLimit >= 0)
);
```

Common data types

Type	Use
CHAR(n)	Fixed-length text
VARCHAR(n)	Variable-length text
INTEGER	Whole numbers
DECIMAL(p,s)	Exact values with precision and scale
DATE / TIMESTAMP	Dates / date and time, depending on DBMS

Keys and constraints

Constraint	Rule
PRIMARY KEY	Unique row identity, with no NULL values
FOREIGN KEY	Reference an existing key
UNIQUE	Prevent duplicate key values
NOT NULL	Require a value
CHECK	Reject a false condition

Defaults and missing values

DEFAULT supplies a value when an INSERT omits a column.
NOT NULL requires a value.
CHECK alone can allow an unknown result caused by NULL.

```
CreditLimit DECIMAL(12,2) DEFAULT 0 NOT NULL
CHECK (CreditLimit >= 0)
```

Foreign keys

A sale can refer to an existing customer.
A nullable reference permits an unidentified cash sale.

```
CREATE TABLE SALE (
  SaleNo INTEGER PRIMARY KEY,
  CustomerNo VARCHAR(9),
  SaleDate DATE NOT NULL,
  Amount DECIMAL(12,2) NOT NULL,
  FOREIGN KEY (CustomerNo)
    REFERENCES CUSTOMER(CustomerNo)
);
```

ALTER TABLE and DROP TABLE

ALTER changes an existing definition.
DROP removes the table and its data.
Dependencies can prevent a change or removal.

```
ALTER TABLE EMPLOYEE ADD ManagerID INTEGER;

ALTER TABLE EMPLOYEE
ADD CONSTRAINT FK_MANAGER
FOREIGN KEY (ManagerID)
REFERENCES EMPLOYEE(EmployeeID);

DROP TABLE SALE;
```

INSERT

List the destination columns explicitly.
Values match the columns by position.
Text uses single quotes. NULL has no quotes.

```
INSERT INTO CUSTOMER
  (CustomerNo, First, Last, CreditLimit)
VALUES
  ('1113-7741', 'Wayne', 'Jones', 0);
```

UPDATE

SET defines the new values. WHERE selects the rows.
Without WHERE, every row is eligible for the update.

```
UPDATE CUSTOMER
SET CreditLimit = 5000
WHERE CustomerNo = '1113-7741';

UPDATE PRODUCT
SET Price = Price * 1.10;
```

DELETE

DELETE removes rows and keeps the table.
DROP removes the table itself.
Verify the intended rows with SELECT before changing data.

```
DELETE FROM SALE
WHERE SaleNo = 12345;

-- Without WHERE, all rows are eligible
DELETE FROM SALE;
```

SELECT and FROM

SELECT chooses output columns. FROM identifies the source.
Explicit columns keep application queries clear.

```
SELECT CustomerNo, First, Last
FROM CUSTOMER;

-- All columns
SELECT * FROM CUSTOMER;
```

Duplicates and DISTINCT

SQL results can contain repeated rows.
DISTINCT compares every selected value together.

```
SELECT Salesrep FROM SALE;

SELECT DISTINCT Salesrep FROM SALE;

SELECT DISTINCT Salesrep, SaleDate
FROM SALE;
```

WHERE and comparisons

WHERE keeps rows whose condition is true.

```
SELECT Description, Price
FROM PRODUCT
WHERE Price > 10;
```

Operators	Meaning
=, <>	Equal, not equal
<, <=	Less than, less than or equal
>, >=	Greater than, greater than or equal

AND, OR, and NOT

AND requires both conditions. OR requires at least one.
AND has higher precedence than OR.
Parentheses make the intended grouping explicit.

```
WHERE (CreditLimit = 0 OR CustomerNo IS NULL)
  AND SaleDate >= '2002-08-01'
```

BETWEEN, IN, and LIKE

```
WHERE Price BETWEEN 10 AND 20

WHERE Last IN ('Jones', 'Adams')

WHERE Last LIKE 'Jo%'
```

Predicate	Meaning
BETWEEN	Inclusive lower and upper bounds
IN	Membership in a list
LIKE	Pattern matching: % any length, _ one character

NULL and unknown values

NULL represents missing, unknown, or inapplicable data.
Zero, empty text, and a space have different meanings.
Ordinary comparisons with NULL produce unknown.
WHERE excludes both false and unknown results.

```
WHERE CustomerNo IS NULL

WHERE CustomerNo IS NOT NULL
```

ORDER BY

Results have no guaranteed order without ORDER BY.
Multiple sort columns apply in the listed sequence.
ASC sorts ascending. DESC sorts descending.

```
SELECT First, Last
FROM CUSTOMER
ORDER BY Last, First;

SELECT SaleNo, Amount
FROM SALE
ORDER BY Amount DESC, SaleNo ASC;
```

INNER JOIN

Only matching rows survive.
A sale without an identified customer has no match.

```
SELECT S.SaleNo, S.Amount, C.First, C.Last
FROM SALE S
JOIN CUSTOMER C
  ON S.CustomerNo = C.CustomerNo;
```

LEFT JOIN

Every sale remains, including cash sales.
Unmatched customer columns contain NULL in the result.
A later WHERE filter can still remove those rows.

```
SELECT S.SaleNo, C.First, C.CreditLimit
FROM SALE S
LEFT JOIN CUSTOMER C
  ON S.CustomerNo = C.CustomerNo;
```

Join types

Join	Rows retained
INNER	Matching rows
LEFT	Matches and unmatched left rows
RIGHT	Matches and unmatched right rows
FULL OUTER	Matches and unmatched rows from either side
CROSS	Every combination of rows

Join cardinality

One customer with three sales produces three joined rows.
Repeated customer details can be correct.
A wrong join condition can multiply rows and inflate totals.
Always define what one result row represents.

Always define what one result row represents.

Aggregate functions

Function	Calculation
COUNT(*)	All result rows
COUNT(column)	Non-null values
SUM(column)	Total of non-null values
AVG(column)	Average of non-null values
MIN / MAX	Smallest / largest non-null value

NULL in aggregates

Example inputs: 10, 20, NULL
AVG divides by the number of non-null values.

Expression	Result
COUNT(*)	3
COUNT(Value)	2
SUM(Value)	30
AVG(Value)	15

GROUP BY

Each group produces one result row.
Group by the customer number to calculate customer totals.

```
SELECT CustomerNo,
       COUNT(*) AS NumSales,
       SUM(Amount) AS TotalSales
FROM SALE
GROUP BY CustomerNo;
```

Before and after grouping

Two sales for A become one customer total.

Input customer	Input amount	Grouped customer	Total
A	100	A	250
A	150	B	200
B	200		

Selected columns in grouped queries

For portable SQL, select grouping expressions and aggregates.
Some products recognize functional dependencies.

```
SELECT C.CustomerNo, C.First, C.Last,
       COUNT(*) AS NumSales
FROM SALE S
JOIN CUSTOMER C
  ON S.CustomerNo = C.CustomerNo
GROUP BY C.CustomerNo, C.First, C.Last;
```

WHERE and HAVING

WHERE filters individual rows before grouping.
HAVING filters groups after aggregation.

```
SELECT CustomerNo, SUM(Amount) AS TotalSales
FROM SALE
WHERE Amount > 0
GROUP BY CustomerNo
HAVING SUM(Amount) > 2000;
```

Counting optional matches

A customer without sales still has a preserved joined row.
COUNT(S.SaleNo) counts actual sales and returns zero.

```
SELECT C.CustomerNo, COUNT(S.SaleNo) AS NumSales
FROM CUSTOMER C
LEFT JOIN SALE S
  ON C.CustomerNo = S.CustomerNo
GROUP BY C.CustomerNo;
```

Comments and expressions

A comment beginning with -- continues to the line end.
Calculated columns exist in the query result.

```
-- Stock above or below the reorder level
SELECT Description,
       QtyInStock - ReorderLevel AS QtyRemaining
FROM PRODUCT;
```

Constants and aliases

A constant appears in each result row.
Table aliases shorten references. Column aliases name outputs.
Aliases leave stored tables and data unchanged.

```
SELECT S.Salesrep,
       'has sold to' AS Relationship,
       C.First, C.Last
FROM SALE S
JOIN CUSTOMER C
  ON S.CustomerNo = C.CustomerNo;
```

Self-join: employees and managers

EMP and MGR represent different roles of the same table.
A LEFT JOIN also retains employees without a manager.

```
SELECT EMP.Name AS EmployeeName,
       MGR.Name AS ManagerName
FROM EMPLOYEE EMP
JOIN EMPLOYEE MGR
  ON EMP.ManagerID = MGR.EmployeeID;
```

Set operations

Operation	Result for {1,2,3} and {2,3,4}
UNION	1, 2, 3, 4
UNION ALL	1, 2, 3, 2, 3, 4
INTERSECT	2, 3
First EXCEPT second	1
Second EXCEPT first	4

Compatible query results

Both branches need the same number of columns.
Corresponding positions need compatible data types.
Names do not determine compatibility.
UNION removes duplicate complete rows.
UNION ALL retains all rows.

UNION ALL retains all rows.

Customer counts and cash counts

The first branch counts sales per identified customer.
The second produces one total for all cash sales.

```
SELECT C.CustomerNo, C.First AS FirstName,
       C.Last AS LastName, COUNT(*) AS NumSales
FROM SALE S JOIN CUSTOMER C
  ON S.CustomerNo = C.CustomerNo
GROUP BY C.CustomerNo, C.First, C.Last
UNION
SELECT ' ', ' ', 'Cash sale', COUNT(*)
FROM SALE WHERE CustomerNo IS NULL
ORDER BY LastName, FirstName;
```

One cash-sales summary row

Five unidentified sale records produce a count of five.
This counts sales, not identified customers.

CustomerNo	FirstName	LastName	NumSales
(space)	(space)	Cash sale	5

Building a query incrementally

Identify tables and join conditions.
Test row filters separately, then combine them.
Add grouping and group filters when needed.
Choose output expressions and sorting.
Compare the result with a known expected answer.

Compare the result with a known expected answer.

Logical query processing

Stage	Purpose
FROM and JOIN	Build the row source
WHERE	Filter rows
GROUP BY and HAVING	Build and filter groups
SELECT and DISTINCT	Produce output and optional deduplication
ORDER BY	Sort the final result

Daily non-credit sales

Business question
What is the total sales value for each day in August 2002?
Include customers with zero credit and unidentified cash sales.
These categories follow the example’s business rule.

These categories follow the example’s business rule.

Credit limit and sales value

Field or expression	Meaning
CUSTOMER.CreditLimit	Maximum credit allowed to the customer
CreditLimit = 0	Customer has no credit facility
SALE.Amount	Value of one sale
SUM(SALE.Amount)	Total value of selected sales

Why the example needs LEFT JOIN

The source contains seven sales.
INNER JOIN matches only six sales to customers.
LEFT JOIN preserves the unidentified cash sale.
The non-credit filter then selects the intended five sales.

```
WHERE (C.CreditLimit = 0
       OR S.CustomerNo IS NULL)
```

The August date range

An exclusive upper boundary includes all of August.
It also covers times throughout August 31.

```
S.SaleDate >= '2002-08-01'
AND S.SaleDate < '2002-09-01'
```

Complete daily-total query

Assumption: SaleDate contains dates without times.

```
SELECT S.SaleDate, SUM(S.Amount) AS TotalSales
FROM SALE S
LEFT JOIN CUSTOMER C
  ON S.CustomerNo = C.CustomerNo
WHERE (C.CreditLimit = 0
       OR S.CustomerNo IS NULL)
  AND S.SaleDate >= '2002-08-01'
  AND S.SaleDate < '2002-09-01'
GROUP BY S.SaleDate
ORDER BY S.SaleDate;
```

Daily totals from the book

Each row summarizes the qualifying sales for one day.

SaleDate	TotalSales
2002-08-12	167.65
2002-08-13	5120.25
2002-08-14	2327.34

Stored procedures

A stored procedure can accept a start and end date.
The database stores the routine for reuse.
Applications call it with parameters.
Security still depends on permissions and safe SQL handling.

Security still depends on permissions and safe SQL handling.

Correctness and performance

Verify join cardinality before calculating totals.
Retrieve the rows and columns the application needs.
Test missing values and date boundaries.
Inspect expensive joins on large tables.
Precomputed summaries require storage and refresh rules.

Precomputed summaries require storage and refresh rules.

Interview answer: joins and aggregation

An INNER JOIN returns matches. A LEFT JOIN also preserves left rows.
WHERE filters rows before grouping. HAVING filters groups.
COUNT(*) counts rows. COUNT(column) counts non-null values.
Join cardinality determines whether a total is meaningful.

Join cardinality determines whether a total is meaningful.

SQL in a real project

A reporting endpoint accepts a start date and an end date.
The application binds typed query parameters.
The database returns daily totals for the chosen period.
Tests cover cash sales, zero credit, and date boundaries.

Tests cover cash sales, zero credit, and date boundaries.

Chapter summary

SQL defines structures and retrieves or changes data.
Join types and NULL handling change the meaning of results.
Grouping turns selected rows into useful business summaries.
Key Takeaway: test every query against an expected result.

Key Takeaway: test every query against an expected result.

Created with AI assistance. Reviewed and presented by Abdalla Fadl.